

# Basics of Complexity Analysis: The RAM Model and the Growth of Functions

Dr. Aimal Tariq Rextin

# Review :The Fibonacci Sequence

■ The sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

■ Mathematical definition:  $F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$

# Review :The Fibonacci Sequence

- The sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

- Mathematical definition: 
$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$$

- Implementation on a computer:

```
FIBONACCI(n)
1  if n == 0
2      return 0
3  elseif n == 1
4      return 1
5  else return FIBONACCI(n - 1) + FIBONACCI(n - 2)
```

# Complexity of Our First Algorithm

- We need a mathematical characterization of the performance of the algorithm

We'll call it the algorithm's *computational complexity*

# Complexity of Our First Algorithm

- We need a mathematical characterization of the performance of the algorithm

We'll call it the algorithm's *computational complexity*

- Let  $T(n)$  be the number of *basic steps* needed to compute  $\text{FIBONACCI}(n)$

# Complexity of Our First Algorithm

- We need a mathematical characterization of the performance of the algorithm

We'll call it the algorithm's *computational complexity*

- Let  $T(n)$  be the number of *basic steps* needed to compute  $\text{FIBONACCI}(n)$

```
FIBONACCI( $n$ )  
1  if  $n == 0$   
2      return 0  
3  elseif  $n == 1$   
4      return 1  
5  else return FIBONACCI( $n - 1$ ) + FIBONACCI( $n - 2$ )
```

# Complexity of Our First Algorithm

- We need a mathematical characterization of the performance of the algorithm

We'll call it the algorithm's *computational complexity*

- Let  $T(n)$  be the number of *basic steps* needed to compute  $\text{FIBONACCI}(n)$

```
FIBONACCI( $n$ )  
1  if  $n == 0$   
2      return 0  
3  elseif  $n == 1$   
4      return 1  
5  else return FIBONACCI( $n - 1$ ) + FIBONACCI( $n - 2$ )
```

$$T(0) = 2; T(1) = 3$$

# Complexity of Our First Algorithm

- We need a mathematical characterization of the performance of the algorithm

We'll call it the algorithm's *computational complexity*

- Let  $T(n)$  be the number of *basic steps* needed to compute  $\text{FIBONACCI}(n)$

```
FIBONACCI( $n$ )  
1  if  $n == 0$   
2      return 0  
3  elseif  $n == 1$   
4      return 1  
5  else return FIBONACCI( $n - 1$ ) + FIBONACCI( $n - 2$ )
```

$$T(0) = 2; T(1) = 3$$

$$T(n) = T(n - 1) + T(n - 2) + 4$$

# Complexity of Our First Algorithm (2)

- But  $T(n) = T(n - 1) + T(n - 2) + 4$  does not give us much information.

Let us try to derive a simpler equation that although is not very accurate is more meaningful.

# Complexity of Our First Algorithm (2)

- But  $T(n) = T(n-1) + T(n-2) + 4$  does not give us much information.

Let us try to derive a simpler equation that although is not very accurate is more meaningful.

Let us assume that  $T(n-1) \approx T(n-2)$

# Complexity of Our First Algorithm (2)

- But  $T(n) = T(n-1) + T(n-2) + 4$  does not give us much information.

Let us try to derive a simpler equation that although is not very accurate is more meaningful.

Let us assume that  $T(n-1) \approx T(n-2)$

Hence  $T(n) = T(n-1) + T(n-2) + 4$  can now be written as

$$T(n) = T(n-1) + T(n-1) + 4$$

$$T(n) = 2T(n-1) + c, \text{ where } c \text{ is a constant equal to } 4$$

# Complexity of Our First Algorithm (2)

- But  $T(n) = T(n-1) + T(n-2) + 4$  does not give us much information.

Let us try to derive a simpler equation that although is not very accurate is more meaningful.

Let us assume that  $T(n-1) \approx T(n-2)$

Hence  $T(n) = T(n-1) + T(n-2) + 4$  can now be written as

$$T(n) = T(n-1) + T(n-1) + 4$$

$T(n) = 2T(n-1) + c$ , where  $c$  is a constant equal to 4

$$T(n) = 2T(n-1) + c = 2[T(n-2) + T(n-3) + c] + c = 4T(n-2) + 3c$$

# Complexity of Our First Algorithm (2)

- But  $T(n) = T(n-1) + T(n-2) + 4$  does not give us much information.

Let us try to derive a simpler equation that although is not very accurate is more meaningful.

Let us assume that  $T(n-1) \approx T(n-2)$

Hence  $T(n) = T(n-1) + T(n-2) + 4$  can now be written as

$$T(n) = T(n-1) + T(n-1) + 4$$

$T(n) = 2T(n-1) + c$ , where  $c$  is a constant equal to 4

$$T(n) = 2T(n-1) + c = 2[T(n-2) + T(n-3) + c] + c = 4T(n-2) + 3c$$

$$T(n) = 4T(n-2) + 3c = 4[T(n-3) + T(n-4)] + 3c = 8T(n-3) + 7$$

# Complexity of Our First Algorithm (2)

- But  $T(n) = T(n-1) + T(n-2) + 4$  does not give us much information.

Let us try to derive a simpler equation that although is not very accurate is more meaningful.

Let us assume that  $T(n-1) \approx T(n-2)$

Hence  $T(n) = T(n-1) + T(n-2) + 4$  can now be written as

$$T(n) = T(n-1) + T(n-1) + 4$$

$T(n) = 2T(n-1) + c$ , where  $c$  is a constant equal to 4

$$T(n) = 2T(n-1) + c = 2[T(n-2) + T(n-3) + c] + c = 4T(n-2) + 3c$$

$$T(n) = 4T(n-2) + 3c = 4[T(n-3) + T(n-4)] + 3c = 8T(n-3) + 7$$

We can repeat this .... and the general form of the formula is

$$T(n) = 2^k T(n-k) + (2^k - 1)c$$

# Complexity of Our First Algorithm (2)

- But  $T(n) = T(n-1) + T(n-2) + 4$  does not give us much information.

Let us try to derive a simpler equation that although is not very accurate is more meaningful.

Let us assume that  $T(n-1) \approx T(n-2)$

Hence  $T(n) = T(n-1) + T(n-2) + 4$  can now be written as

$$T(n) = T(n-1) + T(n-1) + 4$$

$T(n) = 2T(n-1) + c$ , where  $c$  is a constant equal to 4

$$T(n) = 2T(n-1) + c = 2[T(n-2) + T(n-3) + c] + c = 4T(n-2) + 3c$$

$$T(n) = 4T(n-2) + 3c = 4[T(n-3) + T(n-4)] + 3c = 8T(n-3) + 7$$

We can repeat this .... and the general form of the formula is

$T(n) = 2^k T(n-k) + (2^k - 1)c$  We know that  $T(0) = 2$  hence by replacing  $n-k=0$  or  $n=k$  we get

$$2^n T(0) + (2^n - 1)c = 2^n * 2 + C$$

# Complexity of Our First Algorithm (2)

- But  $T(n) = T(n-1) + T(n-2) + 4$  does not give us much information.

Let us try to derive a simpler equation that although is not very accurate is more meaningful.

Let us assume that  $T(n-1) \approx T(n-2)$

Hence  $T(n) = T(n-1) + T(n-2) + 4$  can now be written as

$$T(n) = T(n-1) + T(n-1) + 4$$

$T(n) = 2T(n-1) + c$ , where  $c$  is a constant equal to 4

$$T(n) = 2T(n-1) + c = 2[T(n-2) + T(n-3) + c] + c = 4T(n-2) + 3c$$

$$T(n) = 4T(n-2) + 3c = 4[T(n-3) + T(n-4)] + 3c = 8T(n-3) + 7$$

We can repeat this ... and the general form of the formula is

$$T(n) = 2^k T(n-k) + (2^k - 1)c$$

We know that  $T(0) = 2$  hence by replacing  $n-k=0$  or  $n=k$  we get

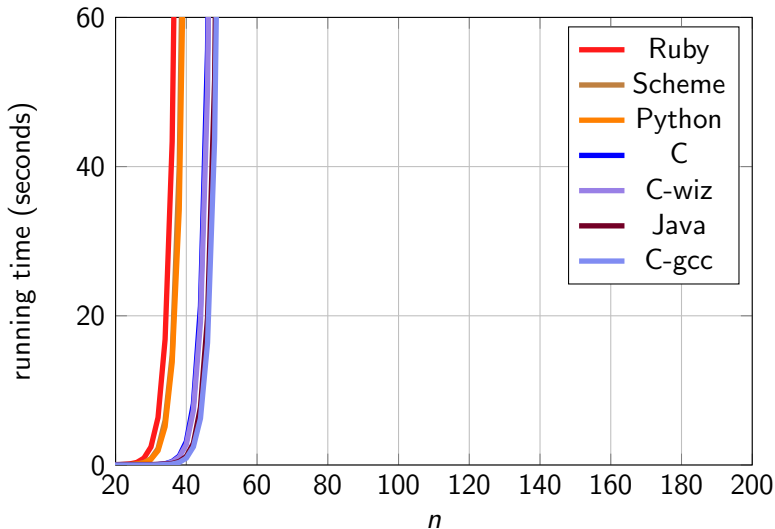
$$2^n T(0) + (2^n - 1)c = 2^n * 2 + C$$

$$T(n) \propto 2^n$$

- Can we do better?

# Running Time

$T(n) \propto 2^n$  i.e Running time is directly proportional an exponential function



# A Better Algorithm

- Again, the sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

# A Better Algorithm

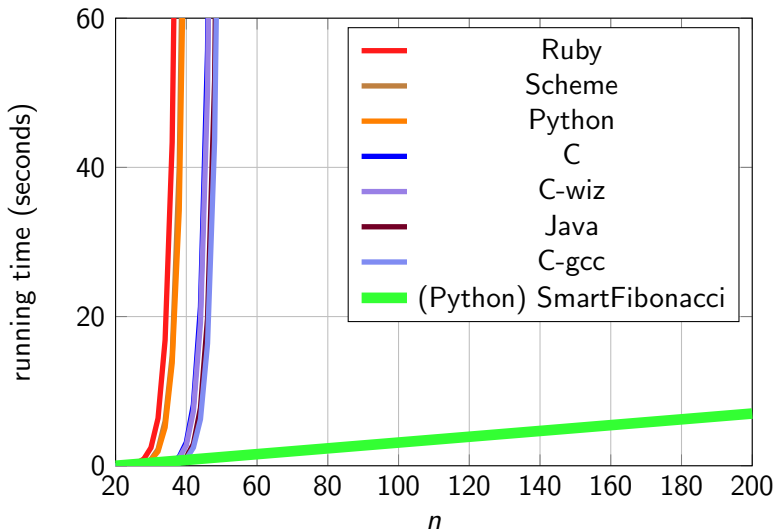
- Again, the sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
- Idea: we can build  $F_n$  from the ground up!

# A Better Algorithm

- Again, the sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
- Idea: we can build  $F_n$  from the ground up!

```
SMARTFIBONACCI(n)  
1  if n == 0  
2      return 0  
3  elseif n == 1  
4      return 1  
5  else pprev = 0  
6      prev = 1  
7      for i = 2 to n  
8          f = prev + pprev  
9          pprev = prev  
10         prev = f  
11 return f
```

# Results



# Complexity of SMARTFIBONACCI

SMARTFIBONACCI( $n$ )

```
1  if  $n == 0$ 
2      return 0
3  elseif  $n == 1$ 
4      return 1
5  else  $prev = 0$ 
6       $pprev = 1$ 
7      for  $i = 2$  to  $n$ 
8           $f = prev + pprev$ 
9           $pprev = prev$ 
10          $prev = f$ 
11 return  $f$ 
```

# Complexity of SMARTFIBONACCI

```
SMARTFIBONACCI(n)
1  if n == 0
2      return 0
3  elseif n == 1
4      return 1
5  else prev = 0
6      pprev = 1
7      for i = 2 to n
8          f = prev + pprev
9          pprev = prev
10         prev = f
11 return f
```

$T(n) =$

# Complexity of SMARTFIBONACCI

```
SMARTFIBONACCI(n)  
1  if n == 0  
2      return 0  
3  elseif n == 1  
4      return 1  
5  else prev = 0  
6      pprev = 1  
7      for i = 2 to n  
8          f = prev + pprev  
9          pprev = prev  
10         prev = f  
11 return f
```

$$T(n) = 6 + 6(n - 1)$$

# Complexity of SMARTFIBONACCI

```
SMARTFIBONACCI(n)  
1  if n == 0  
2      return 0  
3  elseif n == 1  
4      return 1  
5  else prev = 0  
6      pprev = 1  
7      for i = 2 to n  
8          f = prev + pprev  
9          pprev = prev  
10         prev = f  
11 return f
```

$$T(n) = 6 + 6(n - 1) = 6n$$

# Complexity of SMARTFIBONACCI

```
SMARTFIBONACCI(n)
1  if n == 0
2      return 0
3  elseif n == 1
4      return 1
5  else prev = 0
6      pprev = 1
7      for i = 2 to n
8          f = prev + pprev
9          pprev = prev
10         prev = f
11 return f
```

$$T(n) = 6 + 6(n - 1) = 6n$$

The *complexity* of SMARTFIBONACCI(*n*) is *linear* in *n*

# Complexity of SMARTFIBONACCI

SMARTFIBONACCI( $n$ )

```
1  if  $n == 0$ 
2      return 0
3  elseif  $n == 1$ 
4      return 1
5  else  $prev = 0$ 
6       $pprev = 1$ 
7      for  $i = 2$  to  $n$ 
8           $f = prev + pprev$ 
9           $pprev = prev$ 
10          $prev = f$ 
11 return  $f$ 
```

# Complexity of SMARTFIBONACCI

```
SMARTFIBONACCI(n)  
1  if n == 0  
2      return 0  
3  elseif n == 1  
4      return 1  
5  else prev = 0  
6      pprev = 1  
7      for i = 2 to n  
8          f = prev + pprev  
9          pprev = prev  
10         prev = f  
11 return f
```

$T(n) =$

# Complexity of SMARTFIBONACCI

```
SMARTFIBONACCI(n)  
1  if n == 0  
2      return 0  
3  elseif n == 1  
4      return 1  
5  else prev = 0  
6      pprev = 1  
7      for i = 2 to n  
8          f = prev + pprev  
9          pprev = prev  
10         prev = f  
11 return f
```

$$T(n) = c_1 + c_2n$$

# Complexity of SMARTFIBONACCI

```
SMARTFIBONACCI(n)  
1  if n == 0  
2      return 0  
3  elseif n == 1  
4      return 1  
5  else prev = 0  
6      pprev = 1  
7      for i = 2 to n  
8          f = prev + pprev  
9          pprev = prev  
10         prev = f  
11 return f
```

$$T(n) = c_1 + c_2n \text{ hence } T(n) \propto n$$

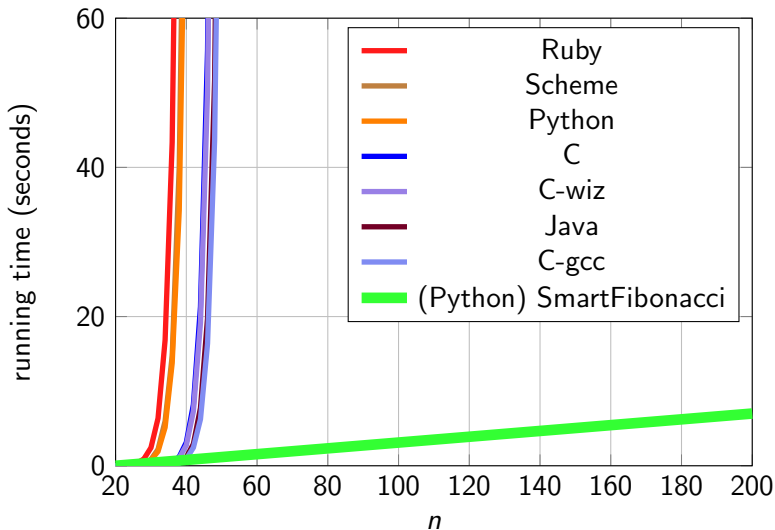
# Complexity of SMARTFIBONACCI

```
SMARTFIBONACCI(n)  
1  if n == 0  
2      return 0  
3  elseif n == 1  
4      return 1  
5  else prev = 0  
6      pprev = 1  
7      for i = 2 to n  
8          f = prev + pprev  
9          pprev = prev  
10         prev = f  
11 return f
```

$T(n) = c_1 + c_2n$  hence  $T(n) \propto n$

The *complexity* of SMARTFIBONACCI(*n*) is *linear* in *n*

# Results



# Slow vs. Fast Fibonacci

- We informally characterized our two Fibonacci algorithms

# Slow vs. Fast Fibonacci

- We informally characterized our two Fibonacci algorithms
  - ▶ FIBONACCI is *exponential* in  $n$
  - ▶ SMARTFIBONACCI is (almost) *linear* in  $n$

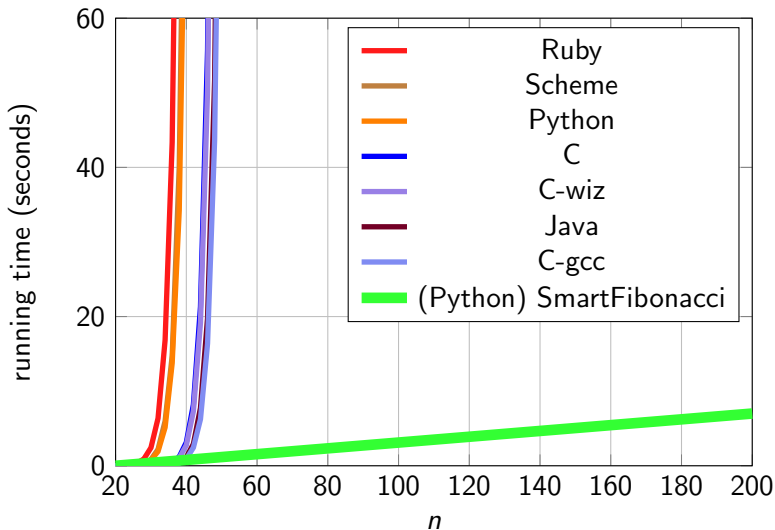
# Slow vs. Fast Fibonacci

- We informally characterized our two Fibonacci algorithms
  - ▶ FIBONACCI is *exponential* in  $n$
  - ▶ SMARTFIBONACCI is (almost) *linear* in  $n$
- How do we characterize the complexity of algorithms?
  - ▶ in general

# Slow vs. Fast Fibonacci

- We informally characterized our two Fibonacci algorithms
  - ▶ FIBONACCI is *exponential* in  $n$
  - ▶ SMARTFIBONACCI is (almost) *linear* in  $n$
- How do we characterize the complexity of algorithms?
  - ▶ in general
  - ▶ in a way that is *specific to the algorithms*
  - ▶ but *independent of implementation details*

# Slow versus Fast Fibonacci Algorithm



# A Model of the Computer

- An informal model of the *random-access machine (RAM)*

# A Model of the Computer

- An informal model of the *random-access machine (RAM)*
- *Basic types* in the RAM model

# A Model of the Computer

- An informal model of the *random-access machine (RAM)*
- *Basic types* in the RAM model
  - ▶ integer and floating-point numbers
  - ▶ limited size of each “word” of data (e.g., 64 bits)

# A Model of the Computer

- An informal model of the *random-access machine (RAM)*
- *Basic types* in the RAM model
  - ▶ integer and floating-point numbers
  - ▶ limited size of each “word” of data (e.g., 64 bits)
- *Basic steps* in the RAM model

# A Model of the Computer

- An informal model of the *random-access machine (RAM)*
- *Basic types* in the RAM model
  - ▶ integer and floating-point numbers
  - ▶ limited size of each “word” of data (e.g., 64 bits)
- *Basic steps* in the RAM model
  - ▶ *operations involving basic types*
  - ▶ load/store: assignment, use of a variable
  - ▶ arithmetic operations: addition, multiplication, division, etc.
  - ▶ branch operations: conditional branch, jump
  - ▶ subroutine call

# A Model of the Computer

- An informal model of the *random-access machine (RAM)*
- *Basic types* in the RAM model
  - ▶ integer and floating-point numbers
  - ▶ limited size of each “word” of data (e.g., 64 bits)
- *Basic steps* in the RAM model
  - ▶ *operations involving basic types*
  - ▶ load/store: assignment, use of a variable
  - ▶ arithmetic operations: addition, multiplication, division, etc.
  - ▶ branch operations: conditional branch, jump
  - ▶ subroutine call
- A *basic step* in the RAM model takes a *constant time*

# Analysis in the RAM Model

```
SMARTFIBONACCI(n)
1  if n == 0
2      return 0
3  elseif n == 1
4      return 1
5  else pprev = 0
6      prev = 1
7      for i = 2 to n
8          f = prev + pprev
9          pprev = prev
10         prev = f
11 return f
```

# Analysis in the RAM Model

SMARTFIBONACCI( $n$ )

```
1  if  $n == 0$ 
2      return 0
3  elseif  $n == 1$ 
4      return 1
5  else  $p_{prev} = 0$ 
6       $prev = 1$ 
7      for  $i = 2$  to  $n$ 
8           $f = prev + p_{prev}$ 
9           $p_{prev} = prev$ 
10          $prev = f$ 
11 return  $f$ 
```

$cost$      $times (n > 1)$

# Analysis in the RAM Model

SMARTFIBONACCI( $n$ )

```
1  if  $n == 0$ 
2      return 0
3  elseif  $n == 1$ 
4      return 1
5  else  $pprev = 0$ 
6       $prev = 1$ 
7      for  $i = 2$  to  $n$ 
8           $f = prev + pprev$ 
9           $pprev = prev$ 
10          $prev = f$ 
11 return  $f$ 
```

| <i>cost</i> | <i>times</i> ( $n > 1$ ) |
|-------------|--------------------------|
| $c_1$       | 1                        |
| $c_2$       | 0                        |
| $c_3$       | 1                        |
| $c_4$       | 0                        |
| $c_5$       | 1                        |
| $c_6$       | 1                        |
| $c_7$       | $n$                      |
| $c_8$       | $n - 1$                  |
| $c_9$       | $n - 1$                  |
| $c_{10}$    | $n - 1$                  |
| $c_{11}$    | 1                        |

$$T(n) = c_1 + c_3 + c_5 + c_6 + c_{11} + nc_7 + (n - 1)(c_8 + c_9 + c_{10})$$

# Analysis in the RAM Model

SMARTFIBONACCI( $n$ )

```
1  if  $n == 0$ 
2      return 0
3  elseif  $n == 1$ 
4      return 1
5  else  $pprev = 0$ 
6       $prev = 1$ 
7      for  $i = 2$  to  $n$ 
8           $f = prev + pprev$ 
9           $pprev = prev$ 
10          $prev = f$ 
11 return  $f$ 
```

| <i>cost</i> | <i>times</i> ( $n > 1$ ) |
|-------------|--------------------------|
| $c_1$       | 1                        |
| $c_2$       | 0                        |
| $c_3$       | 1                        |
| $c_4$       | 0                        |
| $c_5$       | 1                        |
| $c_6$       | 1                        |
| $c_7$       | $n$                      |
| $c_8$       | $n - 1$                  |
| $c_9$       | $n - 1$                  |
| $c_{10}$    | $n - 1$                  |
| $c_{11}$    | 1                        |

$$T(n) = nC_1 + C_2 \Rightarrow T(n) \text{ is a linear function of } n$$

# Input Size

- In general we measure the complexity of an algorithm *as a function of the **size** of the input*
  - ▶ size of the input, can be measured as number of elements in an array, or number of bits
  - ▶ depends what size in a given problem

# Input Size

- In general we measure the complexity of an algorithm *as a function of the **size** of the input*
  - ▶ size of the input, can be measured as number of elements in an array, or number of bits
  - ▶ depends what size in a given problem
  - ▶ did we do that for SMARTFIBONACCI?

# Input Size

- In general we measure the complexity of an algorithm *as a function of the **size** of the input*
  - ▶ size of the input, can be measured as number of elements in an array, or number of bits
  - ▶ depends what size in a given problem
  - ▶ did we do that for SMARTFIBONACCI?
- **Example:** given a sequence  $A = \langle a_1, a_2, \dots, a_n \rangle$ , and a value  $x$ , output TRUE if  $A$  contains  $x$

# Input Size

- In general we measure the complexity of an algorithm *as a function of the size of the input*
  - ▶ size of the input, can be measured as number of elements in an array, or number of bits
  - ▶ depends what size in a given problem
  - ▶ did we do that for SMARTFIBONACCI?
- **Example:** given a sequence  $A = \langle a_1, a_2, \dots, a_n \rangle$ , and a value  $x$ , output TRUE if  $A$  contains  $x$

```

FIND( $A, x$ )
1  for  $i = 1$  to  $\text{length}(A)$ 
2      if  $A[i] == x$ 
3          return TRUE
4  return FALSE

```

# Input Size

- In general we measure the complexity of an algorithm *as a function of the size of the input*
  - ▶ size of the input, can be measured as number of elements in an array, or number of bits
  - ▶ depends what size in a given problem
  - ▶ did we do that for SMARTFIBONACCI?
- **Example:** given a sequence  $A = \langle a_1, a_2, \dots, a_n \rangle$ , and a value  $x$ , output TRUE if  $A$  contains  $x$

```

FIND( $A, x$ )
1  for  $i = 1$  to  $\text{length}(A)$ 
2      if  $A[i] == x$ 
3          return TRUE
4  return FALSE

```

$$T(n) = Cn$$

# Worst-Case Complexity

- In general we measure the complexity of an algorithm *in the worst case*

# Worst-Case Complexity

- In general we measure the complexity of an algorithm *in the worst case*
- **Example:** given a sequence  $A = \langle a_1, a_2, \dots, a_n \rangle$ , output TRUE if  $A$  contains two equal values  $a_i = a_j$  (with  $i \neq j$ )

# Worst-Case Complexity

- In general we measure the complexity of an algorithm *in the worst case*
- **Example:** given a sequence  $A = \langle a_1, a_2, \dots, a_n \rangle$ , output TRUE if  $A$  contains two equal values  $a_i = a_j$  (with  $i \neq j$ )

```
FINDEQUALS(A)
1  for  $i = 1$  to  $\text{length}(A) - 1$ 
2      for  $j = i + 1$  to  $\text{length}(A)$ 
3          if  $A[i] == A[j]$ 
4              return TRUE
5  return FALSE
```

# Worst-Case Complexity

- In general we measure the complexity of an algorithm *in the worst case*
- **Example:** given a sequence  $A = \langle a_1, a_2, \dots, a_n \rangle$ , output TRUE if  $A$  contains two equal values  $a_i = a_j$  (with  $i \neq j$ )

```
FINDEQUALS(A)
1  for i = 1 to length(A) - 1
2      for j = i + 1 to length(A)
3          if A[i] == A[j]
4              return TRUE
5  return FALSE
```

$$T(n) = C\left(\frac{n(n+1)}{2} - n\right)$$

Hence  $T(n) \propto n^2$

# Exercise

- What the worst case running time of the following:

# Exercise

- What the worst case running time of the following:

```
PRINTSOMETHING(n)  
1      i = 1  
2      while i <= n  
3          print Hello  
4          i = i * 2  
5      return
```

Where  $n$  is always a in the form of  $2^k$  where  $k$  is an integer.

# Constant Factors

- Does a load/store operation cost more than, say, an arithmetic operation?

$x = 0$

vs.

$y + z$

# Constant Factors

- Does a load/store operation cost more than, say, an arithmetic operation?

$x = 0$  vs.  $y + z$

- *We do not care about the specific costs of each basic step*
  - ▶ these costs are likely to vary significantly with languages, implementations, and processors
  - ▶ so, we assume  $c_1 = c_2 = c_3 = \dots = c_i$

# Constant Factors

- Does a load/store operation cost more than, say, an arithmetic operation?

$x = 0$

vs.

$y + z$

- *We do not care about the specific costs of each basic step*
  - ▶ these costs are likely to vary significantly with languages, implementations, and processors
  - ▶ so, we assume  $c_1 = c_2 = c_3 = \dots = c_i$
  - ▶ we also ignore the specific *value*  $c_i$ , and in fact *we ignore every constant cost factor*

# Order of Growth

- We care only about the *order of growth* or *rate of growth* of  $T(n)$

# Order of Growth

- We care only about the *order of growth* or *rate of growth* of  $T(n)$ 
  - ▶ so we ignore lower-order terms

E.g., in

$$T(n) = an^2 + bn + c$$

we only consider the  $n^2$  term and say that  $T(n)$  is a quadratic function in  $n$

# Order of Growth

- We care only about the *order of growth* or *rate of growth* of  $T(n)$ 
  - ▶ so we ignore lower-order terms

E.g., in

$$T(n) = an^2 + bn + c$$

we only consider the  $n^2$  term and say that  $T(n)$  is a quadratic function in  $n$

We write

$$T(n) = \Theta(n^2)$$

and say that “ $T(n)$  is theta of  $n$ -squared”